

git add → git commit → git push

format specifier of sizeof : %zu

1 SOS!

Δεν μπορώ να συγκρίνω strings με το "=="! Με αυτό του
ποσο συγκρίνω ανά τα δείκτες τους στη μνήμη (πάντα θα
είναι false - ο το αποτέλεσμα).

Για να συγκρίνω strings μπορώ να χρησιμοποιώ την
συνάρτηση strcmp και να #include τη <string.h>.

2 gcc -Wall -Wextra -Werror file.c -o file

3 αρχεία.h → functions.c περιέχει τις λειτουργίες //include <stdio.h>
functions.h περιέχει τα ορίσματα των λειτουργιών
main.c → #include "functions.h"

Για να τα κανω compile: gcc main.c functions.c -o myprogram

! Όλα τα strings τελειώνουν με '\0'. Αυτόματα!

4 Όταν χρησιμοποιώ την getchar() ή την fgetc() αποθηκεύω
την τιμή επιστροφής του σε μεταβλητή τύπου int και όχι
τύπου char.

5 Αν χρειάζεται και να γράψω και να διαβάσω & είναι
αρχείο χρησιμοποιώ το w+!

6 rewind(fp); → επιστρέφει στην αρχή του αρχείου

7 Σημείο: int main (int argc, char*argv) // int main(int argc, char**argv)

► argv is not a single string; it is an array of strings (pointers)

8 ΠΑΝΤΑ κάνω free() τη μνήμη που δέχεται ένα συνταγμα!
+ Να ελέγχω αν ήλπιξε η malloc!

9 ΠΡΟΣΟΧΗ! Στις λειτουργίες που χειρίζονται pointers και
πινάκες χρησιμοποιώ malloc() για να αποθηκεύσω τα
δεδομένα που μου αρέσουν και να τα χαλώ

Lector

Operating Systems (Λειτουργικά συστήματα)

↳ το λογισμικό του Η/Υ που είναι υπεύθυνο για:

- την διαχείριση του υλικού (συσταμεί) του Η/Υ
- την διαχείριση των πόρων του συστήματος (πόση μνήμη μπορεί να χρησιμοποιηθεί από ένα πρόγραμμα κλπ)
- προσφορά υπηρεσιών σε προγράμματα χρήστη

2 τρόποι διαδραση με Λειτουργικά Συστήματα

► Graphical User interface / Γραφική διασποή χρήση (GUI)

► Command line interface / διασποή γραμμής εντολών, τερματικό, κονσόλα, κώλοα (CLI)

Τεχόνων προγράμματα (εντολές) σε Linux shell

Πρόγραμμα (αριθμο1) Όριστο1 Όριστο2 ... ΌριστοN
αριθμο (argument) / Δεδομένο εισόδου (input data)

► μια τιμή που μπορεί να χρησιμοποιηθεί από το πρόγραμμα κατά το κρίσιμο του

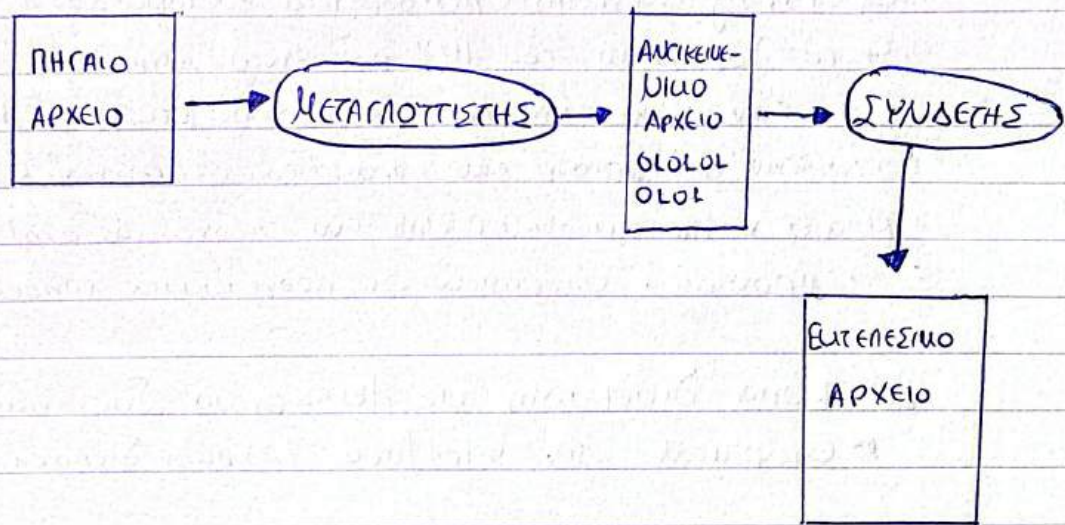
Δεδομένο εξόδου (output)

► η τιμή που επιμερξέει το πρόγραμμα όταν τελειώσει τον υπολογισμό του.

SSH

► το SSH (secure shell) είναι ένα πρωτόκολλο διατύου που επιμερξεί την ασφαλή απομακρυσμένη σύνδεση και διαχείριση ενός υπολογιστή ή server μέσω κρυπτογραφημένης επιμερξωνιάς.

η διαδικασία της μεταγωγισής και σύνδεσης



Κύρια και δευτερεύουσα μνήμη

- ▶ Η δευτερεύουσα μνήμη (secondary memory) αποθηκεύει δεδομένα μόνιμα, δηλαδή σε υλικό που τα διατηρεί ακόμα και χωρίς παροχή ρεύματος (σκληροί δίσκοι, flash drives, ...)
- ▶ Η κύρια μνήμη (primary memory) αποθηκεύει δεδομένα προσωρινά αλλά επιτρέπει στον επεξεργαστή την άμεση πρόσβαση και επεξεργασία τους.

Απολοιδίες Διαφυγής (escape sequences)

- \n ▶ αλλαγή γραμμής
- \r ▶ επαναφορά δρομέα στην αρχή της τρέχουσας γραμμής
- \t ▶ τύπος ενός κενού χώρου ως το tab
- \\ ▶ τύπος αναφοράς μετρίτης (backslash)
- \" ▶ τύπος διηλίου εισαγωγικού
- \\xHH ▶ τύπος του χαρακτήρα που αντιστοιχεί σε HH δεκαδικό
- \\a ▶ δημιουργία ηχητικού σήματος

Τύποι μεταβλητών

Τύπος	συνολικό μέγεθος (bytes)	Εύρος τιμών (min-max)
char	1	-128 ... 127
short int	2	-32.768 ... 32.767
int	4	-2.147.483.648 ... 2.147.483.647
long int	4	-2.147.483.648 ... 2.147.483.647
float	4	Πιθανότητα ακεραιότητας: $1.17 \cdot 10^{-38}$ Πιθανότητα δεκαδικής: $3.4 \cdot 10^{32}$
double	8	Πιθανότητα ακεραιότητας: $2.2 \cdot 10^{-308}$ Πιθανότητα δεκαδικής: $1.8 \cdot 10^{308}$
long double	8, 10, 12, 16	
unsigned char	1	0 ... 255
unsigned short int	2	0 ... 65535
unsigned int	4	0 ... 4.294.967.295
unsigned long int	4	0 ... 4.294.967.295

Συναρτήσεις rand

- Πρέπει να υαρούμε include τα `<stdlib.h>` και `<time.h>`
- Μετά τον main θαρούμε `srand(time(NULL));`
- `arr[i] = rand() % 7` // το αποτέλεσμα είναι τυχαίο αριθμός από το 0 έως το 6
- `arr[i] = rand() % 7 - 3` // το αποτέλεσμα είναι τυχαίο αριθμός στο `[-3, 3]`

lect03 Συναρτήσεις

▶ ανεξάρτητο τμήμα κώδικα που ετερεί μια ορισμένη εργασία και, προαιρετικά, επιστρέφει μία απη.

▶ Η συγγραφή προγραμμάτων με χρήση συναρτήσεων που ετελούν ανεξάρτητες εργασίες είναι η βάση του δομημένου προγραμματισμού

Κλήση συναρτήσεων // function call

```
int g (int x, int y, int z) {  
    return x*x + y*y + z*z + 42;  
}
```

Argument
count

tell me how many command-line arguments
were passed to my program

```
int main (int argc, char **argv) {
```

...
int x = g(1, 2, 3);
...
}

το οποίο του
πεσσεύεται είναι από
τοίς του
ένα όριση

an array of strings
(character pointers)

argv[i] ▶ gives you
the i-th argument

Lect 04

Version control

► Είναι σύστημα που μας επιτρέπει να παρακολουθούμε όλες τις αλλαγές που γίνονται στον κώδικα και να κρατάμε ιστορικό αρχείων.

Γιατί?

► Τα περισσότερα προγράμματα αλλάζουν με τον χρόνο - δεν υπάρχει "τέλυν" μορφή"

► Χάνουμε αλλαγές και κρατάμε μόνο τα αρχεία που χρειάζομαστε χωρίς να χάνουμε πληροφορία

► Μπορούμε να ανατρέξουμε σε οποιαδήποτε αλλαγή έγινε από πάντα στιγμή

Bitwise operators

► Χάνουν μετατροπές στα bit των αλφαριθμητικών.

Τελεστής

Ερμηνεία

Παράδειγμα

&

bitwise AND

$0xFF \& 0xF0 \rightarrow 0xF0$
 $0xFF \& 0x00 \rightarrow 0x00$

|

bitwise OR

$0xFF | 0xF0 \rightarrow 0xFF$
 $0xF0 | 0x0F \rightarrow 0xFF$

^

bitwise XOR

$0xFF \wedge 0xFF \rightarrow 0x00$

$0x00 \wedge 0xFF \rightarrow 0xFF$

~

bitwise NOT

$\sim 0xF0 \rightarrow 0x0F$

$\sim 0x05 \rightarrow 0xFA$

<<

shift left / πολλαπλασιασμός 2^N

$2 \ll 4 \rightarrow 32$

$4 \ll 1 \rightarrow 8$

>>

shift right

$8 \gg 1 \rightarrow 4$

$32 \gg 4 \rightarrow 2$

lectos

Πως απομονώνουμε το πιο σημαντικό bit σε ένα byte;

► byte & 0x80

$$\begin{aligned} \rightarrow 0x80 &= 128_{10} = \\ &= 10000000_2 \end{aligned}$$

Πως δουλεύει το bitwise AND?

$$1 \& 1 \rightarrow 1$$

$$1 \& 0 \rightarrow 0$$

$$0 \& 1 \rightarrow 0$$

$$0 \& 0 \rightarrow 0$$

Ποια η τιμή της παράστασης 0xbeef / 0xcafe0000;

► 0xcafebeef.

Τελεστής συνθήκης (conditional operator)

συνθήκη? τιμή1 : τιμή2

π.χ. (42 > 0) ? 8 : 16 // επιστρέφει 8

Τελεστές αύξησης και μείωσης

Προβλεπόμενα $a++$ → επιστρέφει την τιμή του a ΠΡΙΝ την αύξηση

$a--$ → επιστρέφει την τιμή του a ΠΡΙΝ την μείωση

Επιπλέον $++a$ → επιστρέφει την τιμή του a ΜΕΤΑ την αύξηση

$--a$ → επιστρέφει την τιμή του a ΜΕΤΑ την μείωση

lect 09

camelCase vs snake-case

→ αλφαιμα C → snake-case

```
#include <stdio.h>
```

```
int main() {
```

```
    float a = 3.1;
```

```
    if (a == 3.1)
```

```
        printf("Ok\n");
```

```
    else
```

```
        printf("Not ok\n");
```

```
    return 0;
```

```
}
```

#1 = /one

Not ok

ΓΙΑΤΙ?

a είναι float (32 bit)

3.1 είναι double (64 bit)

binary approximation
differ.

ΔΕΔΟΜΕΝΑ ΕΙΣΟΔΟΥ // input data

• 4 μέθοδοι να εισάγουμε δεδομένα

① Ορίσματα στην γραμμή εντολών

② Γραφοντας κείμενο στην πρότυπη είσοδο
(standard input or stdin) συνήδως με το
πληκτρολόγιο

③ Διαβάζοντας αρχεία από το σύστημα αρχείων

④ Διαβάζοντας από το δίσκο ή αλλού κηρεί

lect 2L

```
char hello[] = "Hello World \n";  
char *world = &hello[6];  
printf ("%s", world);
```

Τονώνει "World \n"

creates an array of characters in memory, including the null terminator `\0` at the end.

`%s` expects a pointer to a null-terminated string. Printing `%s` will start from that pointer and continue until it finds `\0`.

Αναδρομή

► Η συνάρτηση παραγοντικού (Factorial)

- Ο ορισμός του παραγοντικού στα μαθηματικά είναι αναδρομικός (recursive)

$$n! = \begin{cases} 1, & \text{if } n=0 \\ n \times (n-1)!, & \text{if } n>0 \end{cases}$$

Αναδρομή (recursion) είναι η μέθοδος κατά την οποία μια συνάρτηση καλεί τον εαυτό της, με στόχο να επιλύσει ένα υποπροβλήμα του αρχικού προβλήματος, έως ότου φτάσει σε μια βάση τερματισμού (base case) όπου η αναδρομή σταματά.

2 Βασικά στοιχεία

- ① Base Case (Βασική περίπτωση τερματισμού): Μια συνθήκη καθορίζει πότε θα σταματήσει η αναδρομή.
- ② Recursive case (Αναδρομική περίπτωση): Το πώς θα καλέσει τον εαυτό της.

lect 12

! Συναρτηση!

- ▷ συναρτηση atoi (ASCII To Integer)
- ▷ library function in C that converts the numbers in string form to their integer value.

```
int number = atoi(argv[1]);
```

! Δυναμικοί πίνακες με την συναρτηση malloc()

- ▷ με τη βοήθεια δεύτερης μπορούμε να χρησιμοποιήσουμε δυναμικούς πίνακες → πίνακες των οποίων το μέγεθος καθορίζεται δυναμικά → την στιγμή που τρέχει το πρόγραμμα

```
#include <stdlib.h>
```

```
int *array = malloc(N * sizeof(int));
```

Ανοδόξισαν memory με free

```
int *array = malloc(4 * sizeof(int));
```

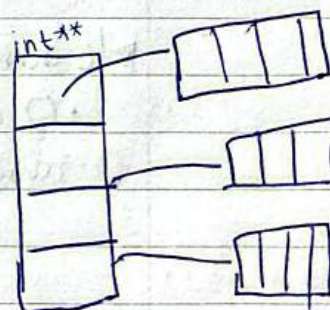
```
if (!array) {
```

```
    // fail gracefully
```

```
}
```

```
free(array); // releases the memory that  
malloc allocated
```

```
if (array == NULL
```



Πίνακες 2 διαστάσεων

```
int rows = 3, cols = 4;
```

```
int *array = (int *) malloc (rows * cols * sizeof(int));
```

```
free(array);
```

↳ typecast

↳ index = (row * width) + cols

lect 13

Υπάρχουν 3 κατηγορίες μνήμης

- ① Η Στοιβα (stack)
- ② Ο Σωρός (heap)
- ③ Η παγκόσμια / στατική μνήμη (global/static memory)

Η στοιβα (stack)

► Συνεχόμενη περιοχή της μνήμης που προστίθενται και αφαιρούνται στοιχεία με σειρά LIFO.

Τι αποθηκεύεται συνήθως στην στοιβα;

- ① Οι τοπικές μεταβλητές που χρησιμοποιεί η κάθε κλήση συνάρτησης
- ② Τα ορίσματα που περνάμε στην κλήση συνάρτησης
- ③ Προσωρινά δεδομένα που αποθηκεύει ο μεταγλωττιστής για κάθε κλήση συνάρτησης

! Για περισσότερα συστήματα, το μέγεθος της στοιβάς είναι περιορισμένο σε μερικά MB - οάτως υπάρχει η προσδοκία ότι δεν θα χάσεις εμφωλευμένες κλήσεις συναρτήσεων ή ως πολύ μεγάλα τοπικά δεδομένα!

`$ulimit -s`
8192

// Μέγιστο
μέγεθος στοιβάς
σε KB, δηλαδή 8MB

Ο Σωρός (heap)

► Ένα σύνολο από τοποθεσίες μνήμης - σε τυχαία σειρά. Σε αντίθεση με τη στοιβα ο σωρός μπορεί να δεσμευτεί ολόκληρη τη διαθέσιμη μνήμη.

Lect 14

Εμβέλεια μεταβλητών (variable scope)

- λεςίται το μέρος του προγράμματος που η μεταβλητή είναι ορατή/προσβάσιμη.

2 είδη μεταβλητών και εμβέλειας

- 1. Τοπικές μεταβλητές (local variables)
- 2. Παγκόσμιες μεταβλητές (global variables)

Τοπικές μεταβλητές

- δηλώνονται μέσα στο σώμα μιας συνάρτησης
- αποθηκεύονται στη σκόιβα
- εμβέλεια: από το σημείο δήλωσής της μέχρι το τέλος του block εντός μέσα στο οποίο ορίστηκε

Παγκόσμιες μεταβλητές

- δηλώνονται έξω από οποιαδήποτε συνάρτηση
- εμβέλεια: από το σημείο δήλωσής της μέχρι το τέλος του αρχείου μέσα στο οποίο ορίστηκε

Στατικές μεταβλητές (static variables)

- Μια μεταβλητή λέγεται στατική (static) όταν διατηρεί την αμνη της αναμνήσε ως κληύει σωμαρτησεω μέχρι το τέλος του προγράμματος. Οι στατικές μεταβλητές αρχυοσασωντά μοίς στην πρώτη κλήση τη σωμαρτησεω.

Παχυόσμια/Στατική μνήμη (Global/Static Memory)

- ▶ Είναι ένα μέρος της μνήμης του προγράμματος ξεχωριστό από την σπείρα και των αμύ που αποθηκεύονται. Παχυόσμια, στατική μεταβλητή, μακρά και δεδομένα του προγράμματος (αριθμοί και ο κώδικας ο ίδιος).
- ▶ Η μνήμη αυτή αξιοποιείται όταν ξεκινά το πρόγραμμα και αποδεσμεύεται όταν τερματίζει.

Συμβολισμός

- ▶ `#include <string.h>`

! Τα string τερματίζονται πάντα με το null byte (0).

ΧΡΗΣΙΜΕΣ ΣΥΝΑΡΤΗΣΕΙΣ

- ▶ `strlen` → Επιστρέφει τον αριθμό των χαρακτήρων για συμβολισμούς χωρίς να μετράει το null byte
- ▶ `strcmp` → Επιστρέφει 0 αν δύο συμβολισμοί A, B είναι ίσοι. Αλλιώς επιστρέφει θετική τιμή αν $A > B$ και αρνητική αν $B > A$.

❗ Ενδιαφερόμαστε πάντα για το σενάριο χειρότερης περίπτωσης! ❗

Lect 15

Πομπηλοσύνη (complexity)

► Η πομπηλοσύνη είναι ένα μέτρο ευαρίνηης τής αποδοσίης αλγορίθμου ή συναρτίης του μέγιστου του προβλήματος που επιλύει

Δύο μετρίες

- ① Χρόνος ευτέλεσης
- ② Χώρος μνήμης που απαιτείται

Χρησιμοποιούμε
συμβολισμούς

Big-O για να
παραφράσμε
"πόσο χειρότερα
μεγαλώνει"

Ο χρόνος/μνήμη
οταν
αυξάνεται
η είσοδος

Πομπηλοσύνη και Big-O Notation

• Ορισμός υλάσεως πομπηλοσύνης

a) Άνω όριο/πόσο κακή μπορεί να ήεί η απόδοση

$$g = O(f) \Leftrightarrow \exists c, \exists n_0 \forall n > n_0: g(n) < c \cdot f(n)$$

b) Κάτω όριο

$$g = \Omega(f) \Leftrightarrow \exists c, \exists n_0 \forall n > n_0: g(n) > c \cdot f(n)$$

c) Ταίρη μέγεθος

$$g = \Theta(f) \Leftrightarrow \exists c_1, c_2, \exists n_0 \forall n > n_0: c_1 \cdot f(n) < g(n) < c_2 \cdot f(n)$$

Διάταξη μετρίων υλάσεως πομπηλοσύνης

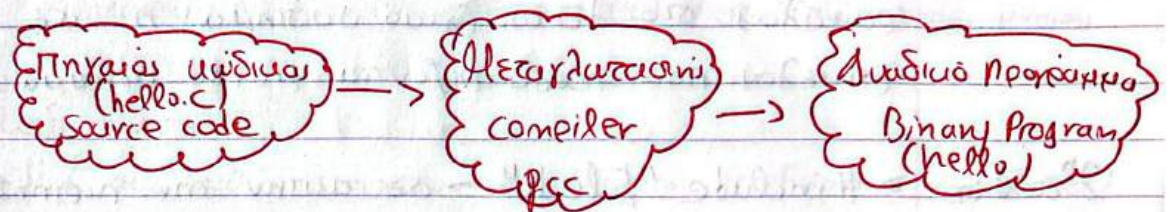
$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

Ασυμπτωτική συμπεριφορά

► Η ασυμπτωτική συμπεριφορά του $f(n) = 2n + 8$ περιγράφεται από τη συναρτίη $f(n) = n$.

Μεταγλωττιστές και Προεπιξεραστής

Μεταγλωττιστής (compiler) είναι ένα πρόγραμμα που μετατρέπει εντολές μιας γλώσσας προγραμματισμού σε ειδικά μηχανή ώστε να μπορεί να διαβαστεί και να τρέξει από τον υπολογιστή.

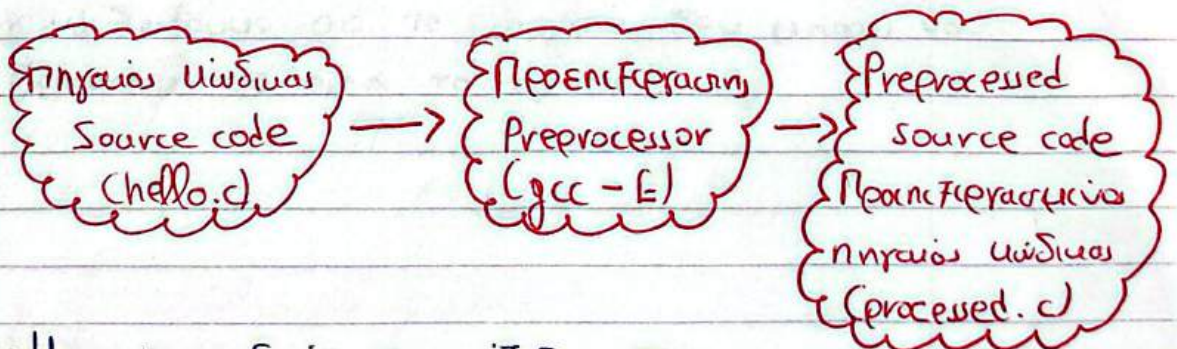


Προεπιξεραστής (preprocessor) είναι ένα υποσύστημα του μεταγλωττιστή.

! Οι εντολές του προεπιξεραστή ξεκινούν με το σύμβολο #!

Ενδεικτικά:

- ① #include
- ② #define
- ③ #if #else #elif #endif
- ④ #ifdef #if #ifndef



► Μπορούμε να δούμε την έξοδο του προεπιξεραστή με το -E (ή τρέχοντας το πρόγραμμα του προεπιξεραστή το ίδιο -cpp):
gcc -E hello.c -o processed.c
ή # cpp hello.c -o processed.c

Η εντολή #include

1^η ευδοκία: ► Η εντολή #include <file.h> εισάγει τα περιεχόμενα του αρχείου file.h* στο σημείο που γραφτηκε το πρόγραμμα.

* Αυτά τα αρχεία βρίσκονται σε προκαθορισμένους φακέλους στο λειτουργικό σύστημα ή σε φακέλους που προσδιορίζονται με το όρισμα -I της gcc.

2^η ευδοκία: ► #include "file.h" - σε αυτήν την περίπτωση ο μεταγλωττιστής ψάχνει πρώτα στον φάκελο που βρίσκεται το πηγαίο αρχείο.

Η εντολή #define

► Η εντολή #define μας επιτρέπει να ορίσουμε μακροεντολές (macros)

1. Ορισμός σταθεράς:

#define TRUE 1

2. Ορισμός υπολογισμού

#define MAX(A,B) ((A) > (B) ? (A) : (B))

Lect 17 Διαδικασία αναζήτησης & Ταξινόησης

Γραμμική/σειριακή αναζήτηση (Linear/serial search)

- αλγόριθμος αναζήτησης ενός στοιχείου σε μια ακολουθία στοιχείων

Χρονική πολυπλοκότητα $O(n)$

Χώρος πολυπλοκότητα $O(1)$ // σταθερός χώρος μνήμης

Ταξινόηση

- Μια ακολουθία στοιχείων a_i λέγεται ταξινομημένη (sorted) ως προς κάποιο τελεστή σύγκρισης $\leq$, αν και μόνο αν $\forall i \leq j$, $a_i \leq a_j$

Διαδικασία αναζήτησης (Binary search)

- αλγόριθμος αναζήτησης που μας επιτρέπει να βρούμε αν ένα στοιχείο βρίσκεται σε μια ακολουθία ταξινομημένων στοιχείων

- Σε κάθε βήμα του αλγόριθμου, η ακολουθία χωρίζεται σε 2 τμήματα και συγκρίνουμε το στοιχείο μας με το μεσαίο στοιχείο της ακολουθίας. Ο αλγόριθμος συνεχίζει σε ένα από τα 2 τμήματα μέχρι να βρεθεί το στοιχείο ή να δείψουμε ότι το στοιχείο δεν μπορεί να βρίσκεται σε αυτό το τμήμα.

Είδη ταξινόμησης

- ▶ Αλγόριθμος επιλογής (selection sort)
- ▶ Αλγόριθμος εισαγωγής (insertion sort)
- ▶ Αλγόριθμος φουσαλίδας (bubble sort).
- ▶ Αλγόριθμος γρήγορης ταξινόμησης (Quick sort).

Αλγόριθμος φουσαλίδας (bubble sort)

```
void bubble_sort (int arr[])
```

```
{
```

```
    int i, j, tmp, reorder;
```

```
    for (i = 1; i < SIZE; i++)
```

```
    {
```

```
        reorder = 0;
```

```
        for (j = SIZE - 1; j >= i; j--)
```

```
        {
```

```
            if (arr[j] < arr[j-1])
```

```
            {
```

```
                /*Ανταλλάσσει τιμές*/
```

```
                tmp = arr[j];
```

```
                arr[j] = arr[j-1];
```

```
                arr[j-1] = tmp;
```

```
                reorder = 1;
```

```
            }
```

```
        }
```

```
        if (reorder == 0)
```

```
            return;
```

```
    }
```

```
}
```


Αλγόριθμος Επιλογής (selection sort)

```
void sel_sort (double arr[])
```

```
{
```

```
    int i, j;
```

```
    double tmp;
```

```
    for (i=0; i< SIZE-1; i++)
```

```
    {
```

```
        for (j=i+1; j<SIZE; j++)
```

```
        {
```

```
            if (arr[i] > arr[j])
```

```
            {
```

```
                /*Αναμετάθεση αριθμών*/
```

```
                tmp = arr[i];
```

```
                arr[i] = arr[j];
```

```
                arr[j] = tmp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

Αλγόριθμος Εισαγωγής (insertion sort)

```
void insertion_sort (int arr[])
```

```
{
```

```
    int i, j, tmp;
```

```
    for (i=1; i<SIZE; i++)
```

```
    {
```

```
        tmp = arr[i];
```

```
        j = i;
```

```
        while (j>0 && (arr[j-1]>tmp))
```

```
        {
```

```
            arr[j] = arr[j-1]; /*Μετακίνηση του στοιχείου
```

```
            κατά μια θέση δεξιά*/
```

```
            j--;
```

```
        } arr[j] = tmp; }
```


! Ξυτελείται ΜΟΛΟ
 ΣΕ ΤΑΞΙΝΟΜΗΜΕΝΟΥΣ
 ΠΙΝΑΚΕΣ !

Αλγόριθμος δυαδικής αναζήτησης (binary search)

Σε κάθε
 επανάληψη
 "υποβούμε"
 τον πίνακα
 στα 2
 ↓

Απογορεύεται
 η πιθανότητα

```
int binary_search (const int arr[], int size, int num)
{
    int start, end, middle;
    start = 0;
    end = size - 1;

    while (start <= end)
    {
        middle = (start + end) / 2;
        if (num < arr[middle])
            end = middle - 1;
        else if (num > arr[middle])
            start = middle + 1;
        else
            return middle;
    }
    return -1;
}
```

→ Αν η ευρεθείσα του
 κωδικού φτάσει σε αυτό
 το σημείο σημαίνει ότι
 ο αριθμός δεν βρέθηκε
 και η συνάρτηση επιστρέφει -1.

Προσέγγιση
 κωδικών:

```
int binary_search (double *array, int size) {
    int low = 0, high = size - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (array[mid] == 0.0) return mid;
        else if (array[mid] < 0.0) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Χρόνος
 πολυπλοκότητα: $O(\log n)$
 (το διάστημα αναζήτησης
 υποδιπλασιάζεται)

Χρόνος
 πολυπλοκότητα: $O(1)$
 (γεννημένος από
 3 απλές μεταβλητές)

!! #include <stdlib.h> !!

Or swapniseu bsearch() uai qsort()

Swapniseu qsort()

```
qsort (array, SIZE, sizeof(int), compare);
```

```
int compare (const void *elem1, const void *elem2)
{
    return *(int *) elem1 - *(int *) elem2;
}
```

typecasting

Swapniseu bsearch()

```
int compare (const void *a, const void *b) {
    return (*(int *) a - *(int *) b);
}
```

: In main

```
int main() {
    int data[] = {1, 12, 27, 42, 89}; // Array must be sorted
    int n = 5;
    int target = 27;
```

```
int *item = bsearch(&target, data, n, sizeof(int), compare);
```

```
if (item != NULL) {
    printf("Found %d at memory address %p\n", *item, (void *) item);
} else {
    printf("Item not found.\n");
}
return 0;
```

```
}
```


Δομές και ενώσεις

Η δήλωση μιας δομής που δεν ακολουθείται από λίστες μεταβλητών δεν προκαλεί δέσμευση μνήμης. Αντί, περιγράφει τη μορφή της δομής.

```
struct Company {  
    char name[50];  
    int start-year;  
    int field;  
    int tax-num;  
    int num-empl;  
    char addr[50];  
    float balance;  
};
```

→ εσωτερική δομή

→ δηλώσεις ηρώων

```
struct Book {  
    char title[100];  
    int year;  
    float price;
```

{b1, b2} — λίστες μεταβλητών — οι μεταβλητές b1, b2 δηλώνονται σαν δομές τύπου Book.

Summario ... structs & typedef.

► struct: lets you group different types of data into one unit

ex.

```
struct Person {  
    int age;  
    float height;  
};
```

to use it →

```
struct Person p1;  
p1.age = 25;  
p1.height = 1.75;
```

► Why structs?

- They represent "real-world" objects
- They keep related data together

Struct + typedef together

► we avoid writing struct every time

ex.

```
typedef struct {  
    int age;  
    float height;  
} Person;
```

usage →

```
Person p1;  
p1.age = 30;  
p1.height = 1.80;
```

SOS
via strings!

ex. `char title[100];` // title is an array
"hello" → a string literal (a pointer to char)
// we can't assign a pointer to an array

Correct way

`strcpy(b1.title, "hello");` // we have to include `<string.h>`

Problem: using "&" with arrays

⚠ Warning ⚠ (Why?)

→ s1.name is an array
arrays always decay to pointers
using & makes the type wrong: char (*)[100]
instead of char*

```
scanf("%s%s", &s1.name, &s2.name);
```

✓ Correct version ✓

```
scanf("%99s%99s", s1.name, s2.name);
```

• width limit (%99s) prevents buffer overflow

Πως δουλεύει το ">" ?

Σε περίπτωση που έχουμε δείκτη σε δομή:

Student *p = &s;

► Δεν μπορούμε να χρησιμοποιήσουμε.

• Πρέπει να χρησιμοποιήσουμε →

* το ">" υπάρχει μόνο για pointers to structs.

Σημείωση - υπενθύμιση για pointers

student (s) the actual struct

student *p = &s;

where s lives in memory

Munipin nou nlares
eva struct

Data structure Alignment (Padding)

- C compilers want the total size of a struct to be a multiple of its largest member's alignment requirement.

char name[50]; → 50 bytes
int AM; → 4 bytes
float grd; → 4 bytes +

S8 ~~X~~ Zu nvu

60 by ks.

```
int maxIndex = 0;
```

```
for(int i=0; i<3; i++){
```

```
scanf ("%s", classroom[i].name); //πρωταρχία στο nido name
```

των 1-0000 μαθητών

```
classroom[i].id = i + 100; // Αυτομάτος αριθμός μπηρανού
```

}

```
for(int i = L; i < 3; i++) {
```

$\text{maxIndex} = i$ // upatixn tn dwin (index) ta maktira

3

3

•

```
return 0;
```

}

Αρχεία στη C

Δεδομένα εισόδου και εξόδου (input and output data)



4 μέθοδοι εισαγωγής δεδομένων

- ▶ Διαβάζονται αρχεία από το σύστημα αρχείων

Αρχείο (file) είναι ένας πόρος για να μεταγράψουμε δεδομένα σε έναν υπολογιστή. Συνήθως αποθηκεύεται στην δευτερεύουσα μνήμη (π.χ. σκληρή δίσκος).

- ▶ Στο Linux σχεδόν τα πάντα είναι ένα αρχείο

Κάθε αρχείο:

- ▶ έχει ένα όνομα (filename/basename)
- ▶ Βρίσκεται μέσα σε ένα συγκευρισμένο φάκελο (directory/folder)
- ▶ έχει ένα πλήρες μονοπάτι (filepath) που καθορίζει που βρίσκεται το αρχείο

Για να δουλέψουμε με αρχεία πρέπει:

① Declare a pointer to a FILE structure
`FILE *fptr;`

② Opening and closing files
`fopen()`, `fclose()`

↓
open a
file

↓
release the memory
once we are done

syntax: `fptr = fopen("filename.txt", "mode");`

Common modes

"r" → Open for reading (fails if file doesn't exist)

"w" → Open for writing (creates a new file or overwrites an existing one)

"a" → Open for appending (adds data to the end of the file)

! Σημείωση !

`size_t` → a data type used to represent the size of objects in memory

► it's the type returned by the `sizeof` operator and is widely used in standard library functions for counting and indexing

Η συνάρτηση `fread`.

Syntax

`size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);`

`ptr`: the address of the memory block (usually an array or a struct) where the data will be stored.

`size`: the size of each element you want to read (in bytes)

`nmemb`: the number of elements you want to read

`stream`: The `FILE` pointer.

`size_t bytesRead = fread(buffer, sizeof(char), 1023, fileToRead);`

ΠΟΣ ΓΡΑΦΟΥΜΕ ΣΕ ΑΡΧΕΙΑ?

► αφού τα ανοίξουμε, χρησιμοποιούμε την εντολή:

`fprintf(fptr, "hello file");`

Η συνάρτηση fwrite

Syntax

size_t fwrite(const void *ptr, size_t size, size_t nmems, FILE *stream)

~~fgets()~~ scanf()

Για να πάρει ονομασία (John Doe)

```
printf("enter full name: ");  
fgets(newStudent.name, 50, stdin);
```

// Remove the trailing newline '\n' added by fgets
newStudent.name[strcspn(newStudent.name, "\n")] = 0;

αν έχουμε την fgets(), ή ή scanf()
πρέπει να χρησιμοποιήσουμε την getchar()
για να καθαρίσει clear την leftover newline
and to buffer.

Συνάρτηση fgetc()

• Πως διαβάζουμε δεδομένα από ένα αρχείο;

```
int counter = 0;
```

```
int ch;
```

```
while ((ch = fgetc(fptr)) != EOF) {  
    counter++;
```

Time complexity
of fgetc: $O(n)$
 $n \rightarrow$ length of input

ΠΩΣ ΔΙΑΒΑΖΟΥΜΕ ΑΝΟ ΑΡΧΕΙΑ?

```
int d;  
fscanf(fptr, "%d", &d);
```


Πεδία Δομών σε Δομές (Struct Bit Fields)

► Ένα πεδίο δομής μπορεί να οριστεί έτσι ώστε να έχει ως μέγεθος στη μνήμη ένα συγκεκριμένο αριθμό **bits**.

Παράδειγμα για να σώσουμε μνήμη.

```
struct student_status {  
    int registered: 1;  
    int year: 3;  
    int grade: 4;  
};
```

```
printf ("%zu\n", sizeof (struct student_status));
```

Αυτοαναφορικές Δομές Self-referential structs

```
struct Student {  
    int id;  
    struct Student *next; // pointer to another student  
};
```

```
int main (void) {  
    struct Student s1, s2;  
    s1.id = 101;  
    s2.id = 102;  
    // Link them together  
    s1.next = &s2;  
    s2.next = NULL; // s2 is the end of the chain
```

// accessing s2 through s1
printf ("%d", s1.next->id);
// to id i.e.: (*s1.next).id
• Get the pointer (address)
 stored inside s1
• Go to that address and look at
 the id field

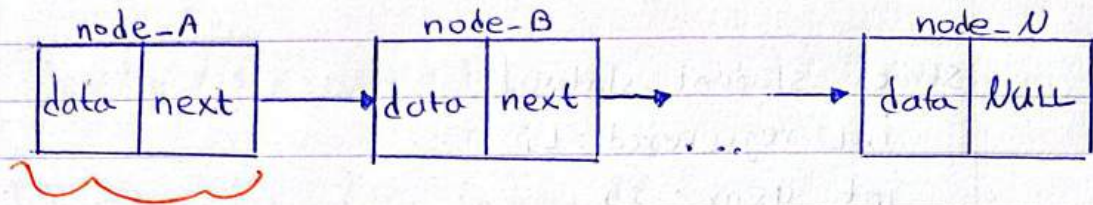
Η αναζήτηση στη λίστα είναι πάντα γραμμική $O(n)$, ένα στον ταξινομημένο πίνακα είναι δυαδική, $O(\log n)$!

Δυναμικές Δομές Δεδομένων

Τα στοιχεία της λίστας είναι διασκορπισμένα στη μνήμη και συνδέονται με pointers.

► **Απλά συνδεδεμένη λίστα (single-linked list)**

► Είναι τύπος δεδομένων που το κάθε στοιχείο δείχνει στο επόμενο και το τελευταίο δείχνει στο NULL.



Κομβός λίστας → αντιπροσωπεύεται από μια δομή που περιέχει τα δεδομένα του κομβού και ένα πεδίο που είναι δείκτης (π.χ. το πεδίο next) στον επόμενο κομβό.

► Ο πρώτος κομβός ονομάζεται κεφαλή (head) της λίστας και ο τελευταίος ουρά (tail).

► Η τιμή του πεδίου δείκτη του τελευταίου κομβού πρέπει να είναι ίση με NULL, ώστε να προσδιορίζεται το τέλος της λίστας.

Περίπτωσης λίστας

► Στοιβά

► Ουρά

Στοιβά

```

typedef struct Node
{
    char name[100];
    int code;
    float grdj;
    struct Node *next;
} Node;
  
```


Δημιουργία λίστας

① Structure of a Node

```
struct Node {  
    int data; // the value  
    struct Node* next; // pointer to the next 'box'  
};
```

head (newan) → points to the first node

last node (oupa) → points to NULL - end of the list

② Add a node at the beginning of the list

```
void insertAtFront (struct Node* - napvaze in stream to pointer. head, int newValue) {  
    // 1 - Prepare the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    newNode->data = newValue; if (newNode == NULL) return;
```

// 2. Link it to the existing list

newNode->next = *head;

// 3. Move the head to point to the new node

*head = newNode;

③ Add a node at the end of the list

```
void add-to-end (struct Node* head, int newValue) {  
    struct Node* newNode = malloc(sizeof(struct Node));  
    newNode->data = newValue;  
    newNode->next = NULL;  
    if (*head == NULL) {  
        *head = newNode;  
        return;  
    }  
    struct Node* temp = *head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    } // Move the address stored in head  
    temp->next = newNode;  
    // the old last node now points  
    // to our new one.
```


Διαχείριση Μνήμης

► Δέσμευση μνήμης

- Στατική
- Δυναμική

► Αποδέσμευση μνήμης

Μνήμη ενός προγράμματος που εκτελείται:

- ① τμήμα κώδικα (code segment) ► χρησιμοποιείται για την αποθήκευση του μεταγλωττισμένου κώδικα του προγράμματος
- ② τμήμα δεδομένων (data segment) ► χρησιμοποιείται για την αποθήκευση των καθολικών και static μεταβλητών του προγράμματος
- ③ Στοιβά (stack) ► αποθήκευση των δεδομένων των συναρτήσεων (π.χ τοπικές μεταβλητές)
- ④ Σωρός (heap) ► χρησιμοποιείται για δυναμική δέσμευση μνήμης

Γενικό
πρόγραμμα

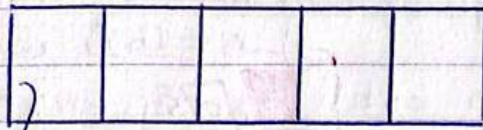
—
Δεν είναι
υποχρεωτικό
για κάθε
σύστημα



Δυναμική Διαχείριση Μνήμης

► η μνήμη δεσμεύεται από τον σωρό κατά την εκτέλεση του προγράμματος και όχι κατά τη μεταγλώττιση

Δυναμικοί πίνακες (malloc())



=> φτιάχνουμε ένα array με pointers



```
int rows = 3;
```

```
int cols = 4;
```

//1. Allocate the array of pointers (rows)

```
int **matrix = malloc(rows * sizeof(int*));
```

```
if (!matrix) { printf("malloc failed\n"); return 1; } //check if malloc failed.
```

//2. For each row, allocate the column

```
for (int i = 0; i < rows; i++) {
```

```
    matrix[i] = malloc(cols * sizeof(int));
```

```
}
```

//Now, we can use like a normal 2d array

matrix[i][j] = 50;

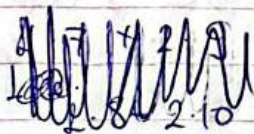
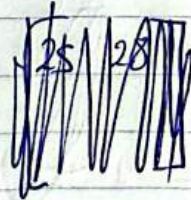
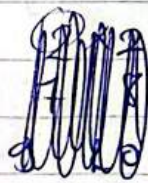
//3. We must free memory in reverse order

```
for (int i = 0; i < rows; i++) {
```

```
    free(matrix[i]); //freeing each row
```

```
}
```

```
free(matrix);
```



Όταν έχουμε $root == NULL$, σημαίνει ότι δεν υπάρχει ούτε ένας κόμβος.

Αν προσπαθήσουμε να κάνουμε $return root \rightarrow value$ ενώ το $root$ είναι $NULL$, θα έχουμε $segmentation fault$.

Δένδρα

→ χρησιμοποιούμε αναδρομή

```
#include <stdio.h>
```

```
int main(void) {
```

```
    struct TreeNode* root;
```

```
    printf("%d\n", sum(root));
```

```
    return 0;
```

```
}
```

```
struct TreeNode {
```

```
    int data;
```

```
    struct TreeNode* left;
```

```
    struct TreeNode* right;
```

```
};
```

```
int sum(struct TreeNode* root) {
```

```
    int x = sum(root->left);
```

```
    int y = sum(root->right);
```

```
    return x + y + root->data;
```

```
}
```

```
if (root == NULL) {
```

```
    return 0;
```

```
} Base
```

```
case
```

```
} Για να
```

```
terminatiou n
```

```
αναδρομή
```

Πολυπλοκότητα: !

Χρονική $O(n)$

όπου n είναι ο
αριθμός των κόμβων

γιατί πρέπει να

επισκεφτούμε κάθε

κόμβο αριβάς μια φορά

Χώρος $O(h)$

όπου h είναι το

ύψος του δένδρου

(λόγω της στροβίλης

αναδρομής)

Πομπηλαιοότητες (extra)

- Ένας απλός βρόχος που διασχίζει στοιχεία δίνει $f(n) = n$.
- Ένας βρόχος μέσα σε έναν βρόχο δίνει $f(n) = n^2$.
- Ένας βρόχος μέσα σε έναν βρόχο μέσα σε ένα βρόχο δίνει $f(n) = n^3$.

Κλήση συνάρτησης εντός βρόχου

```
int i;  
for(i=0; i<n; i++) {  
    f(n);  
}
```

• Αν η $f(n)$ είναι μια συνάρτηση που ετερεί αριθμώς η εντολές, ο αριθμός εντολών όλου του προγράμματος θα είναι ασυμπτωτικά n^2 .

$O(\log n)$

- Δυαδική αναζήτηση
- σημαίνει το πρόβλημα μάς αφορά στη μέση

$O(\sqrt{n})$

- Βρίσκω τους διαιρετές ενός αριθμού (φτάνω μέχρι τη ρίζα του μάς αφορά)

$O(n \log n)$

- merge sort (ο πιο γενικός αλγόριθμος ταξινόμησης)

$O(n^2) - O(n^3)$

- nested loops

$O(n!)$

- permutations

Παράδειγμα Εξέτασής // πιο φτηνό μονοπάτι

map.

L 3 1

2 L 4

4 2 L

► φτιάχνω έναν νέο πίνακα : Ποιό είναι το ελάχιστο κόστος για να φτάσω ΜΕΧΡΙ αυτό το ζευγάρι :

• Πρώτη γραμμή $\rightarrow$ μπορώ να πάω μόνο δεξιά

• Πρώτη στήλη $\rightarrow$ μπορώ να πάω μόνο κάτω

Αρα

L 4 5

3 ? 4 ? 8

7 ? 6 ? 7

• Για να πάω στο (1,1) μπορώ να έρθω είτε από πάνω (4) είτε από αριστερά (3).

$3 < 4$ αρα το νέο κόστος

για το (1,1) είναι

πάλι αμν + 3 = 4

! Σημείωση για ένα αρχείο .txt!

Στη main :

```
FILE *fptr;
```

```
fptr = fopen("map.txt", "r");
```

// ελέγχω για σφάλμα κατά το άνοιγμα του αρχείου

```
if(!fptr) {
```

```
    printf("error opening the file\n");
```

```
    return 1;
```

```
}
```

```
int d = 0;
```

```
fscanf(fptr, "%d", &d);
```


Παράδειγμα επέκτασης // Αντιστροφή λίστας

```
typedef struct node {  
    int value;  
    struct node * next;  
} *List;
```

Το δείκτη μας `head`, ουσιαστικά, να αλλάζουμε την κατεύθυνση όλων των δεικτών `next`.

π.χ

Αν έχουμε $[1] \rightarrow [2] \rightarrow [3] \rightarrow \text{NULL}$

πρέπει να γίνει $\text{NULL} \leftarrow [1] \leftarrow [2] \leftarrow [3]$ // το `head` να δείχνει στο 3.

Υλοποίηση

Χρησιμοποιάμε 3 βοηθητικούς δείκτες

- ① `prev` : δείχνει στον προηγούμενο κόμβο (ξεκινάει ως `NULL`)
- ② `curr` : Ο κόμβος στον οποίο βρίσκουμε τώρα (ξεκινάει από το `head`).
- ③ `next` : Χρησιμοποιείται για να "απαραχαιρέσει" τον επόμενο κόμβο πριν αλλάξουμε τη φορά του δείκτη, για να μην τον χάσουμε.

Ο κώδικας:

```
List reverse_list (List head) {
```

```
    struct node *prev = NULL;
```

```
    struct node *curr = head;
```

```
    struct node *next = NULL;
```

```
    while (curr != NULL) {
```

```
        next = curr->next;
```

```
        curr->next = prev;
```

```
        prev = curr;
```

```
        curr = next;
```

```
    }
```

```
    return prev;
```

```
}
```

// Μετακινούμε τους δείκτες `curr` και `prev` μπροστά

Παράδειγμα εφάρμοξης // Αθροισμή δένδρων

```
typedef struct node {  
    int value;  
    struct node *left;  
    struct node *right;  
} *Tree;
```

```
int sumtree (Tree root) {
```

// Βασική αναδρομή

```
    if (root == NULL) {
```

```
        return 0;
```

```
    }
```

// Αναδρομικό Βήμα: τρέχουσα τιμή + τιμή αριστερά + τιμή δεξιά

```
    return root->value + sumtree (root->left) + sumtree (root->right);
```

```
}
```


! Για nίκες structs...

Έστω ότι έχουμε

```
for(int i=0; i<S;i++){  
    strcpy(warehouse[i].name, name[i]);  
    warehouse[i].price = p[i];  
}
```

► Όταν γράφουμε `warehouse[i]` εννοούμε (`warehouse + i`)

• Δηλ. ο compiler βρίσκει τη διεύθυνση του i-οστού στοιχείου αλλά δεν αλλάζει την τιμή που είναι αποθηκευμένη μέσα στη μεταβλητή `warehouse`.

► Η `warehouse` συνεχίζει να δείχνει σταθερά στην αρχή (στο στοιχείο 0).

Αντιμετάθεση αριθμών με χρήση συναρτήσεων

```
void change(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
int main(void) {  
    int a=5;  
    int b=6;  
    change(&a, &b);  
}
```